

# 杂项

Morning Flower and Evening Oath

XCPC vector 模板分册

# 目录

|                   |    |
|-------------------|----|
| 使用约定              | ii |
| 第一章 离线算法          | 1  |
| 1.1 莫队算法          | 1  |
| 1.1.1 带修改莫队       | 1  |
| 1.1.1.1 带修改的莫队    | 1  |
| 第二章 配套知识与验证范围     | 3  |
| 2.1 带修莫队：时间轴与区间状态 | 3  |

# 使用约定

C++20，默认包含 `bits/stdc++.h` 并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

# 第一章 离线算法

## 1.1 莫队算法

### 1.1.1 带修改莫队

#### 1.1.1.1 带修改的莫队

ModifiedMo( $n, m$ ) 保存离线询问,  $n > 0$ ,  $m$  为修改总数。add( $l, r, t$ ) 登记第  $t$  次修改完成后的  $[l, r]$ , 返回从 0 开始的询问编号;  $0 \leq l \leq r \leq n$ ,  $0 \leq t \leq m$ , 询问数小于 INT\_MAX。run(add, del, chg, ans, block=0) 排序并移动三个指针。add( $p$ )/del( $p$ ) 按下标更新当前区间统计; chg( $i, forward, l, r$ ) 将编号  $i$  的修改前进或回退, 并同步修正当前  $[l, r]$  内的贡献; ans( $id$ ) 记录结果。修改编号从 0 开始, 前进第  $i$  次修改时原时间为  $i$ , 回退时原时间为  $i+1$ 。回调初始状态必须为原数组、空区间、时间 0; 重复 run 前由调用方重置状态。状态不会自动恢复, 原询问顺序不变。仅适用于可增删维护的区间统计, 需区分左右端操作时另行拆分回调。默认块长约  $n^{2/3}$ , 也可显式指定正块长。 $n$ 、询问数、修改数同阶且回调  $O(1)$  时总时间  $O(n^{5/3})$ , 调度器空间  $O(q)$ 。

```
1 struct ModifiedMo
2 {
3     struct Query
4     {
5         int l, r, t, id;
6     };
7
8     int n, m;
9     vector<Query> q;
10
11     ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }
12
13     int add(int l, int r, int t)
14     {
15         assert(0 <= l && l <= r && r <= n);
16         assert(0 <= t && t <= m);
17         int id = q.size();
18         q.push_back({l, r, t, id});
19         return id;
20     }
```

```
22     template <class Add, class Del, class Chg, class Ans>
23     void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
24     {
25         if (q.empty()) return;
26         if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
27         assert(block > 0);
28         auto ord = q;
29         auto cmp = [&](const Query &a, const Query &b)
30         {
31             int x = a.l / block, y = b.l / block;
32             if (x != y) return x < y;
33             int p = a.r / block, q = b.r / block;
34             if (p != q) return (x & 1) ? p > q : p < q;
35             return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
36         };
37         sort(ord.begin(), ord.end(), cmp);
38         int l = 0, r = 0, t = 0;
39         for (auto q : ord)
40         {
41             while (t < q.t)
42             {
43                 chg(t, true, l, r);
44                 t++;
45             }
46             while (t > q.t)
47             {
48                 t--;
49                 chg(t, false, l, r);
50             }
51             while (l > q.l) add(--l);
52             while (r < q.r) add(r++);
53             while (l < q.l) del(l++);
54             while (r > q.r) del(--r);
55             ans(q.id);
56         }
57     }
58 };
```

## 第二章 配套知识与验证范围

### 2.1 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间  $t$  表示前  $t$  次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针  $(l, r, t)$ ，回调统计恰好对应当前版本的半开区间  $[l, r)$ 。前进修改  $i$  时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为  $B$ ，询问数为  $Q$ ，修改数为  $U$ ，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U \min(Q, (\lfloor n/B \rfloor + 1)^2)\right).$$

另有  $O(Q \log Q)$  排序与  $O(Q)$  调度空间。三者同阶时取  $B \approx n^{2/3}$ ，得到  $O(n^{5/3})$ ；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

**P1903 数颜色** 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 **P1903 记录 297653743** 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

# 算法索引

ModifiedMo, 1

带修莫队, 3